

```

# -*- coding: utf-8 -*-
"""
Created on Mon Mar  9 06:41:46 2026

@author: samsc
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, export_graphviz

bank = pd.read_csv('C:/Users/samsc/Desktop/ADS-502-Hands-On-Exercises/bank.csv', sep = ';')
bank

#21. Produce the following graphs. What is the strength of each group? Weakness?

#first build crosstab before plotting
marital_crosstab = pd.crosstab(
    bank['marital'], bank['y'])

#a. Bar graph of marital

bank['marital'].value_counts().plot(kind='bar')

#b.Bar graph of marital with overlay of y

marital_crosstab.plot(kind='bar', stacked=True)

#c.Normalized bar graph of marital, with overlay response.

#To create a normalized version, we need to change the table so that the values
#in each cell are the proportions of "no" and "yes" response values within
#each value of the predictor y
#we do that with the div() function, which divides the values of the crosstab
#table by another object, within each specified axis.
#We want to divide (div name comes from divide) the cells in row 1 of the table
#by the sum of the cells in row 1 and so on(Larose, 2019)

#axis 0 is row
crosstab_mar_norm = marital_crosstab.div(marital_crosstab.sum(1), axis=0)
#then plot it
crosstab_mar_norm.plot(kind='bar', stacked=True)
plt.show()

#The bar graph shows us clearly the differences between the marital status groups.
#It is easy to identify which classes show the most responses to yes and what class
#to target as the bank survey. However just because the number like married marital
#is largest doesn't make it the most likely class to say yes. The normalize data tells
#the story better on which marital status customers to target. However without the counts of
#customers the normalize graph is not useful at confirming if the trend would work
#for a lot of customers.

```

#22. Using the graph from Exercise 21c, describe the relationship between marital status and response.

#As we see here, the married category by percentage is least likely to say yes, despite the high number of married people in this data set. Single or divorced would be the better targeted audience for the bank subscription.

#23. Do the following with variables marital and response.

#a. Build a contingency table, being careful to have the correct variables representing the rows and columns. Report the counts and the column percentages.

#Best practices: Contingency Tables

#Let the response variable represent the rows.

#Then, obtain the column percentages to directly compare the response proportions

#for each category of the predictor

#Getting the counts

```
crosstab_mar_contin = pd.crosstab(
    bank['y'], bank['marital'])
```

```
print(crosstab_mar_contin)
```

#axis = 1 is the columns

```
crosstab_mar_contin_percentage = round(crosstab_mar_contin.div(crosstab_mar_contin.sum(0), axis = 1)
```

```
print(crosstab_mar_contin_percentage)
```

#b. Describe what the contingency table is telling you.

#With the contingency table being shown in counts and percentages, the percentages contingency table proved to tell the most useful information. It shows what the bar graph shows but more clearly. The divorced marital status is the category with the most likelihood to response with yes, meaning that the bank customer target should be on divorced status people instead of married single people.

#24. Repeat the previous exercise, this time reporting the row percentages. Explain the difference between the interpretation of this table and the previous contingency table.

```
crosstab_mar_contin_row_per = round(crosstab_mar_contin.div(crosstab_mar_contin.sum(1), axis = 0) * 100
print(crosstab_mar_contin_row_per)
```

#The contingency table with the focus on row percentages has the percentages add up by row to 100% instead of by column, giving a different interpretation. What this contingency table is describing is the amount of people saying no or yes compared to the sum of all the people in the data set. This just shows that the majority of the people who responded to the survey happened to be married, followed by single then lastly divorced. This is not particularly useful information if we are trying to get people to subscribe to the bank subscription.

#25. Produce the following graphs. What is the strength of each graph? Weakness?

#a. Histogram of duration

```
plt.hist(bank['duration'])
```

```
plt.xlabel("Duration in seconds")
```

```
plt.ylabel("Frequency")
```

```
plt.title("Histogram of duration of phone calls")
```

#Heavily right skewed histogram. A majority of the data happens to occur

#between 0 and 500 seconds.

```

#b. Histogram of duration, with overlay of response
bank_duration_y = bank[bank['y'] == 'yes']['duration']
bank_duration_n = bank[bank['y'] == 'no']['duration']

colors = ['orange', 'teal']

plt.hist([bank_duration_y, bank_duration_n], bins=10, stacked=True, color=colors)
plt.legend(['Response = No', "Response = Yes"])
plt.title("Response of saying yes or no to bank subscription based on duration of call")
plt.xlabel('Duration in seconds')
plt.ylabel("Frequency")
plt.show()

#c. Normalized histogram of duration, with overlay of response

#Create stacked histogram but save it in variables

(n, bins, patches) = plt.hist([bank_duration_y, bank_duration_n], bins = 10, stacked = True)

#n is the height of the histogram bars
#bins is the boundaries of each bin in the histogram
#patches is the actual bar objects

#since they are two variables (yes and no) n has two series of numbers. The first number in
#each series is the height of the first bar for each variable. To create a normalized
#histogram, we need to know what proportions of each bin each variable represents. We need
#information contained in n into a matrix to get the column proportions. Combine heights of
#the variables using column_stack() function

plt.figure() #refreshes so it shows proper graph (Spyder feature)

n_table = np.column_stack((n[0], n[1])) #yes and no

#n_table is a two column matrix where each column's entries hold the heights of each bar
#calculating the proportion of each bar using sum method
n_norm = n_table / n_table.sum(axis=1)[:, None] #percentage of yes and no
n_norm = np.nan_to_num(n_norm, nan=0) #deals with division by zero credit to Copilot on this line

#creating upper and lower bounds of each bin
ourbins = np.column_stack((bins[0:10], bins[1:11]))

#Manually creating our barplot using the yes and no bins.
p1 = plt.bar(x = ourbins[:,0], height = n_norm[:,0], width = ourbins[:, 1] - ourbins[:, 0])
p2 = plt.bar(x = ourbins[:,0], height = n_norm[:,1], width=ourbins[:, 1]- ourbins[:, 0], bottom = r
plt.legend(['Response = Yes', 'Response = No'])
plt.title('Normalized Histogram of Duration with Response Overlay')
plt.xlabel('Duration in seconds')
plt.ylabel('Proportion')
plt.show()

#The strengths of the graph is that it proper shows what duration is optimal to getting
#customer responses to say yes. In this case the graph is showing around 2500 seconds to be
#the optimal call. However this is a big issue. Since the amount of calls are not being
#considered, one would think that having a 41 minute call would get someone to say yes.

```

#But, the weakness of this graph is that it is reliant on the counts to verify if the
#call duration is optimal or just that a few customers happened to say yes for that duration
#of the call.

#---Larose(2019) words and code were used here for the most part
`